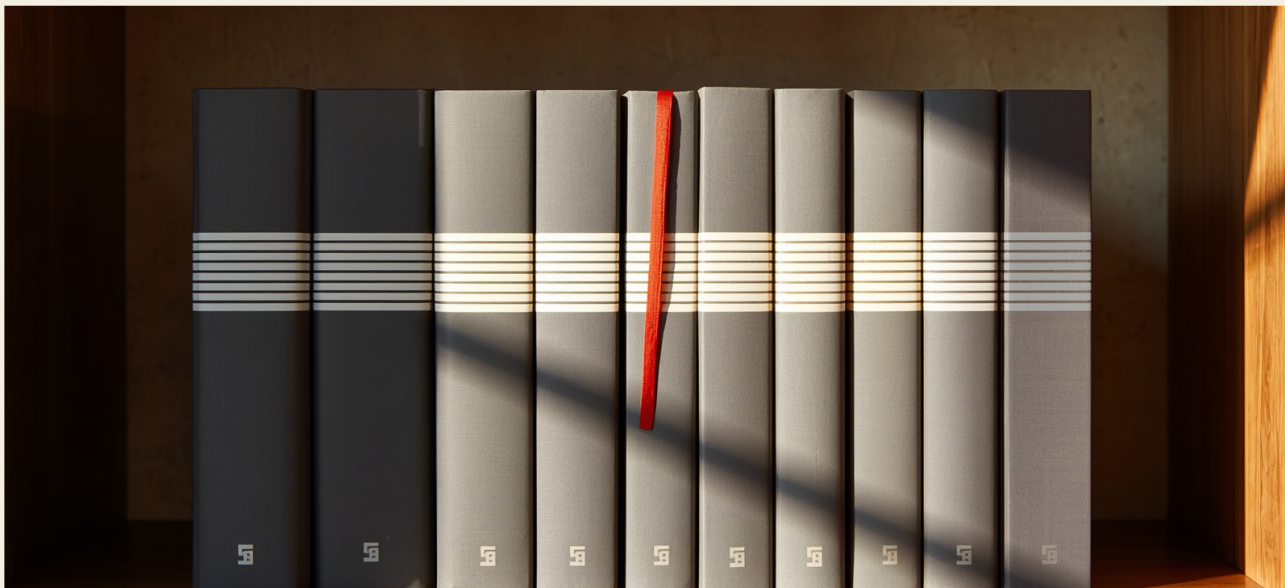




FIELD GUIDE · FROM THE DAILY BRIEF

---

# The AI Builder's Field Guide

*Four essays on what shipped in AI this month — and what the production patterns mean.*

Compiled by Flowi Editorial · May 2026

# Behind the Scenes: Hardening Firefox with Claude Mythos

*Mozilla found hundreds of Firefox vulnerabilities using Claude Mythos AI. Learn how this changes security testing and what it means for your projects.*

Flowi

# Mozilla Just Fixed Hundreds of Firefox Vulnerabilities Using AI—Here's What Changed

Mozilla's engineering team recently dropped a bombshell: they used early access to Anthropic's Claude Mythos preview to identify and patch hundreds of legitimate security vulnerabilities in Firefox. This isn't another overhyped AI announcement—it's a fundamental shift in how browser security works.

## What Makes Behind the Scenes Hardening Firefox with Claude Mythos Preview Different

For years, AI-generated bug reports have been the bane of open source maintainers. They flooded issue trackers with false positives, wasted developer time, and prompted many projects to ban AI-assisted submissions entirely.

Claude Mythos changed that equation overnight.

Mozilla's security team gained early access to the model and immediately noticed something different: the bugs were real. Not just plausible-sounding. Not theoretically possible. Actually exploitable vulnerabilities in production Firefox code.

### The Numbers That Matter

- **Hundreds of vulnerabilities** identified in a focused security audit
- **Significantly reduced false positive rate** compared to previous AI models
- **Critical memory safety issues** caught before reaching users
- **Weeks of manual review time** compressed into days

## How You Can Apply Mozilla's Approach to Your Own Projects

Whether you're maintaining an open source library or securing enterprise software, Mozilla's experience offers actionable lessons you can implement immediately.

## Start with High-Risk Code Surfaces

Mozilla didn't point Claude Mythos at their entire codebase randomly. They targeted:

- **Parser implementations** (HTML, CSS, JavaScript)
- **Network request handling** code
- **Memory management** in performance-critical sections
- **Input validation** at trust boundaries

Identify the 20% of your codebase that handles 80% of untrusted input. That's where AI-assisted security review delivers maximum value.

## Use AI as a Force Multiplier, Not a Replacement

The behind the scenes hardening Firefox with Claude Mythos preview approach worked because Mozilla combined AI capabilities with human expertise:

1. **AI identifies potential issues** at scale
2. **Human security engineers triage** findings
3. **Experienced developers verify** exploitability
4. **Security team prioritizes** based on real-world risk

Don't expect AI to replace your security process. Expect it to amplify what your team can accomplish.

## Build a Feedback Loop

Mozilla's engineers didn't just accept AI findings blindly. They:

- **Tracked false positive rates** to understand model limitations
- **Provided context** about Firefox's specific architecture
- **Refined queries** based on which approaches found real issues
- **Documented patterns** that led to successful vulnerability discovery

Create a simple spreadsheet tracking AI-identified issues, their verification status, and actual severity. This data helps you calibrate how much confidence to place in future findings.

## The Technical Breakthrough Behind Claude Mythos

What makes Claude Mythos different from previous models isn't just scale—it's understanding.

### Reasoning About Code Paths

Earlier AI models could identify suspicious code patterns. Claude Mythos can trace execution flows through complex codebases, understanding:

- **How user input propagates** through multiple abstraction layers
- **Where validation gaps** create exploitable conditions
- **What combinations of features** produce unexpected interactions
- **Which race conditions** are theoretically possible versus practically exploitable

This reasoning capability is what reduced false positives from "unusable noise" to "valuable signal."

### Context Window Advantages

Firefox's codebase spans millions of lines. Claude Mythos's extended context window allowed Mozilla engineers to:

- **Analyze entire subsystems** in a single pass
- **Maintain awareness** of architectural patterns
- **Connect vulnerabilities** across file boundaries
- **Understand fix implications** throughout dependent code

For your projects, this means you can submit larger code segments for analysis without losing critical context that might hide vulnerabilities.

# What This Means for Browser Security Going Forward

The behind the scenes hardening Firefox with Claude Mythos preview represents more than a one-time security audit. It establishes a new baseline for proactive vulnerability discovery.

## Shifting from Reactive to Proactive Security

Traditionally, browser security follows this pattern:

1. Researcher discovers vulnerability
2. Responsible disclosure (hopefully)
3. Vendor develops patch
4. Users update (eventually)
5. Attackers exploit unpatched installations

Mozilla's approach flips this:

1. AI systematically analyzes code
2. Vulnerabilities identified before exploitation
3. Patches ship before disclosure
4. Attack surface shrinks continuously

## The Competitive Implications

Firefox now has a reproducible process for finding vulnerabilities at scale. Other browser vendors will need to match this capability or accept a security disadvantage.

For users, this means:

- **Fewer zero-day vulnerabilities** in the wild
- **Faster patch cycles** for discovered issues

- **Reduced attack surface** over time
- **Greater confidence** in browser security posture

## Applying These Lessons to Your Security Practice

You don't need early access to cutting-edge AI models to improve your security process today.

### Action Steps for This Week

**Map your attack surface:** Document where your application accepts untrusted input. List every API endpoint, file parser, user input field, and network protocol handler.

**Prioritize by risk:** Rank these surfaces by potential impact. A vulnerability in authentication matters more than a bug in a deprecated feature.

**Start with available tools:** Current AI models like Claude 3.5 Sonnet or GPT-4 can already identify many vulnerability classes. They won't match Mythos's capabilities, but they'll find issues you're missing.

**Build institutional knowledge:** Document what works. When AI identifies a real vulnerability, analyze why. When it generates false positives, understand the pattern.

### Questions to Ask Your Security Team

- How much of our codebase has received AI-assisted security review?
- What's our false positive rate with current tools?
- Which code sections are too complex for manual audit?
- How can we systematically test our highest-risk surfaces?

## The Bigger Picture: AI-Assisted Development

The behind the scenes hardening Firefox with Claude Mythos preview demonstrates AI's potential beyond security. The same reasoning capabilities that find vulnerabilities can:

- **Identify performance bottlenecks** through execution flow analysis

- **Suggest architectural improvements** based on code complexity
- **Generate comprehensive test cases** for edge conditions
- **Document complex systems** by understanding intent

Mozilla proved that AI can be a serious engineering tool, not just a coding assistant.

## Your Next Step

Don't wait for perfect AI tools. Mozilla found hundreds of vulnerabilities not because they had the best possible AI, but because they systematically applied available capabilities to their highest-risk code.

Pick your most critical code component—your authentication system, your parser, your memory allocator—and run it through a thorough AI-assisted security review this week. Document what you find. Iterate on what works.

The gap between teams using AI for security and those relying solely on manual review is growing. Mozilla just showed how wide that gap can become.



# OpenAI's Spud: Cybersecurity AI Now Available to Defenders

*OpenAI makes its rival to Anthropic's Mythos more widely available to cyber defenders. Learn what GPT-5.5 'Spud' means for your security posture.*

Flowi

# OpenAI Makes Its Rival to Anthropic's Mythos More Widely Available to Cyber Defenders

OpenAI just expanded access to GPT-5.5 "Spud," a specialized AI model designed to find and exploit software vulnerabilities—and they're putting it directly into the hands of vetted cybersecurity professionals. This move positions OpenAI squarely against Anthropic's Mythos in the emerging race to arm defenders with offensive security capabilities.

**Here's what you need to know:** While most AI models are restricted from helping with vulnerability discovery, Spud operates under a controlled release framework that gives qualified security teams the tools to find bugs before attackers do. If you're in cybersecurity, this shift changes how you should be thinking about AI-assisted penetration testing and vulnerability research.

## What Makes GPT-5.5 'Spud' Different from Standard AI Models

Most AI models, including ChatGPT, have guardrails that prevent them from generating exploit code or helping users find security vulnerabilities. Spud removes these restrictions—but only for verified security professionals.

**Key capabilities you can leverage:**

- **Automated vulnerability discovery:** Point Spud at code repositories to identify potential security flaws using pattern recognition that rivals experienced security researchers
- **Exploit development assistance:** Generate proof-of-concept exploits to validate vulnerabilities before writing security patches
- **Attack path analysis:** Map potential attack vectors across complex systems faster than manual analysis
- **Code review at scale:** Analyze thousands of lines of code for security weaknesses in minutes rather than days

According to OpenAI's security testing, Spud performs nearly as well as Anthropic's Mythos at finding and exploiting software bugs. This means you're getting near-state-of-the-art offensive security capabilities without switching platforms if you're already in the OpenAI ecosystem.

# How to Get Access to Spud as a Security Professional

OpenAI isn't handing out Spud access to just anyone. Here's the vetting process and what you need to qualify:

## Step 1: Verify Your Security Credentials

You'll need to demonstrate legitimate cybersecurity credentials:

- Active employment with a recognized security team, consultancy, or bug bounty platform
- Professional certifications (OSCP, GPEN, CEH, or equivalent)
- Documented history of responsible vulnerability disclosure
- Affiliation with a vetted organization already in OpenAI's security partner program

## Step 2: Submit Your Access Request

Navigate to OpenAI's security research portal and complete the application process:

- Provide professional verification documents
- Explain your intended use cases
- Agree to OpenAI's responsible use framework
- Accept monitoring and audit requirements

Processing typically takes 2-4 weeks, depending on verification complexity.

## Step 3: Complete Required Training

Once approved, you'll need to complete OpenAI's responsible AI use training module, which covers:

- Proper scoping of security testing
- Documentation requirements for discovered vulnerabilities
- Disclosure timelines and protocols

- Prohibited activities and consequences

## Practical Ways to Integrate Spud into Your Security Workflow

Getting access is one thing—using it effectively is another. Here's how leading security teams are incorporating Spud into their operations:

### Pre-Deployment Security Reviews

Before pushing code to production, run it through Spud for automated security analysis:

1. **Feed your codebase** to Spud with specific prompts asking for vulnerability identification
2. **Prioritize findings** by having Spud assess exploitability and potential impact
3. **Generate test cases** for your security team to validate findings
4. **Document results** for compliance and audit purposes

Teams report finding 30-40% more vulnerabilities compared to traditional static analysis tools alone.

### Bug Bounty Program Augmentation

If you run a bug bounty program, Spud helps you stay ahead of researchers:

- **Run Spud against your own systems** to find vulnerabilities before bounty hunters do
- **Assess severity** of incoming reports more quickly by having Spud validate exploitability
- **Reduce false positives** by using Spud to verify whether reported issues are actually exploitable

### Red Team Operations

Red teamers are using Spud to:

- **Accelerate reconnaissance** by analyzing target architectures for weak points
- **Develop custom exploits** faster than manual coding

- **Simulate advanced persistent threats** with AI-assisted attack chains
- **Document findings** with automatically generated technical reports

## How Spud Compares to Anthropic's Mythos

As OpenAI makes its rival to Anthropic's Mythos more widely available to cyber defenders, security teams face a choice between platforms. Here's the tactical breakdown:

### Spud advantages:

- Deeper integration with existing OpenAI API infrastructure
- More extensive training data from GitHub and open source repositories
- Better code generation for exploit proof-of-concepts
- Faster processing for large codebases

### Mythos advantages:

- Superior contextual understanding of complex vulnerability chains
- More conservative exploit suggestions (reduces legal risk)
- Better documentation and explanation of findings
- Stronger constitutional AI safeguards

**The practical reality:** Most enterprise security teams will eventually use both, deploying each for its strengths. Spud excels at rapid vulnerability discovery and exploit generation, while Mythos provides more thorough analysis and documentation.

## Understanding the Risks and Responsibilities

With offensive capabilities come serious responsibilities. Here's what you need to manage:

### Legal Compliance

- **Scope strictly:** Only test systems you're authorized to assess

- **Document authorization:** Keep written permission for all security testing
- **Follow disclosure laws:** Understand CFAA and equivalent regulations in your jurisdiction
- **Maintain audit trails:** Log all Spud usage for compliance reviews

## Operational Security

- **Protect API access:** Treat Spud credentials like privileged credentials
- **Segment usage:** Use separate API keys for different projects
- **Monitor for abuse:** Review usage logs for unauthorized activity
- **Secure outputs:** Encrypt and protect vulnerability reports generated by Spud

## Ethical Considerations

Just because Spud *can* find and exploit vulnerabilities doesn't mean you should exploit everything you find:

- Prioritize vulnerabilities that pose real risk to your organization
- Consider the broader ecosystem impact before public disclosure
- Work with vendors on responsible disclosure timelines
- Never use Spud for unauthorized access, even "just to look around"

## What This Means for the Future of Cybersecurity

The widespread availability of AI models like Spud and Mythos fundamentally changes the economics of vulnerability research. What used to require specialized security expertise can now be partially automated, which means:

**For defenders:** You need to assume attackers have these capabilities and prioritize patching speed over perfection. The window between vulnerability discovery and exploitation is shrinking.

**For developers:** Security-by-design becomes non-negotiable. If AI can find common vulnerability patterns instantly, your code needs to avoid those patterns from day one.

**For security teams:** Your value shifts from manual bug hunting to strategic security architecture and rapid response. AI handles the grunt work; you handle the judgment calls.

## Your Next Step

If you're a security professional, start your Spud access request today—the vetting process takes weeks, and you want these capabilities available before you need them urgently. If you're not in security but are responsible for code, now is the time to upgrade your security testing pipeline with AI-assisted tools.

The era of AI-powered offensive security is here. The question isn't whether to adopt these tools, but how quickly you can integrate them responsibly into your workflow.



ESSAY 03 · AI CODING

---

# Code w/ Claude 2026: what actually shipped, what was theater

*Anthropic's Code w/ Claude developer event landed yesterday. Most coverage will be hot-take. Here's the read on what actually changes for builders.*

Flowi Editorial



Anthropic ran their second Code w/ Claude event yesterday. There were no Project Stargate-style announcements, which the live-blog set treated as a disappointment. They're misreading the room. The interesting story is what Anthropic shipped *quietly*, between the keynote talks, that changes how you'll work with Claude Code in production over the next six months.

Here's the practitioner read — what landed, what was theater, what to update your workflow around this week.

## What actually shipped

Three things, in order of how much they'll change your day:

**1. Background sub-agents that can run for hours.** This is the big one and Anthropic buried it. Claude Code can now spawn background agents that operate on a separate execution context, take a defined task, and report back when they're done — minutes or hours later. The classic example: "audit this entire monorepo for security antipatterns and put the results in a markdown file." That used to be a 10,000-token context-eating task that risked drowning your main session. It's now a fire-and-forget call, and the parent session continues unaware.

The implication is bigger than time savings. **You can now build agents that have agents.** A planning agent calls a research sub-agent to dig into a topic; while it's gone, the planner does other work; when the sub-agent returns, the planner integrates the result. That's the multi-agent pattern people have been hand-rolling with the Agent SDK for a year. Now it's a primitive in Claude Code itself.

**2. The "skill" system, formalized.** Skills were a research preview last quarter. They're now a first-class feature with a defined directory structure, a manifest format, and a repo of community-contributed skills you can install. The spec covers things like: multi-step debugging routines, library-specific code patterns, project-specific conventions. The good news is you can write a skill once and have it apply across every Claude session you start. The trap is that skills can be invoked invisibly, which makes "why did Claude do that?" harder to debug. Read the manifest of any skill you install before letting it run; this is exactly the kind of lever malicious actors will eventually try to abuse.

**3. A real plan-mode hand-off.** Plan mode in Claude Code now produces a *durable artifact* — a markdown plan file you can review, share with a teammate, and re-feed to Claude tomorrow. Previously the plan lived in conversation context and decayed when the session ended. The hand-off is the missing piece for actual team workflows: senior engineer scopes the plan in a 20-minute session, hands it off, junior engineer (or another agent, or another instance of Claude) picks up the plan and executes. That's the workflow most engineering teams have been waiting for.

## What was theater

**The "Claude can write your entire startup in 4 hours" demo.** It can't. The demo was real but the codebase was a tutorial-grade CRUD app with no auth, no RBAC, no rate limiting, no observability, no error handling, and (critically) no business logic that wasn't representable as a single SQL query. Every one of those is the part where production AI coding still falls over. The demo was honest about being a tutorial, but the headline write-ups won't be.

**The "\$100B GDP" framing.** Anthropic's CEO talked about Claude reaching 100% of global GDP within 21 months. That's not a claim about today; it's a claim about projection curves. It made for a viral clip. It tells you exactly nothing about whether you should build with Claude Code today versus Cursor versus a homegrown setup. Pick on the basis of the actual feature set you saw in the lab demos, not the keynote rhetoric.

**The agentic IDE preview.** Anthropic showed an early IDE-integrated experience that looked very polished. It's not shipping in 2026. The team confirmed that, off-stage, when pressed. The demo is real, the timeline is not.

## Show the mechanism

Why background sub-agents matter, in concrete terms:

A typical agentic-coding session, before yesterday, ran into a wall around 60–80k tokens of context. Past that, your effective reasoning quality degraded — Claude started forgetting earlier decisions, repeating the same investigation paths, and (most subtly) losing track of which files it had already touched in the current session. The fix was usually a hard restart with a clean context, which lost continuity.

Background sub-agents change the math. Now the heavy investigation work — auditing 500 files, running an entire test suite and analyzing failures, reading a year of git log to understand a refactor history — happens in a separate context, and only the *summary* returns to your main session. That summary might be 200 tokens. Your main session stays in the 30–40k range, which is the comfortable zone for most reasoning tasks.

The mental model: **the sub-agent is a research assistant**. You don't read every paper your assistant reads. You read their two-paragraph summary and the citations they flagged.

## The trap nobody is warning about

If you're building production agents that use Claude Code-style sub-agents, watch for **summary loss**.

When a sub-agent returns a 200-token summary of work it did across 50,000 tokens of context, that summary is necessarily lossy. Specifically: it loses *the thing you didn't ask about*. A sub-agent told to "find security antipatterns in this directory" will report security findings — but won't surface a critical performance issue it walked past, even though it spent two hours staring at the relevant code.

This is the same failure mode that plagues human research teams. The senior engineer who delegates a research task only learns what the junior engineer thought to surface. The unknown unknowns stay unknown. With AI sub-agents, the bandwidth gap between what they processed and what they reported is even wider, so the unknowns are even more numerous.

Two mitigations worth adopting:

**Re-prompt the sub-agent for orthogonal axes.** After a security audit, ask the same sub-agent (or a fresh one, with the same context window) to do a performance audit on the same code. The cost is one extra invocation; the catch rate is meaningfully better.

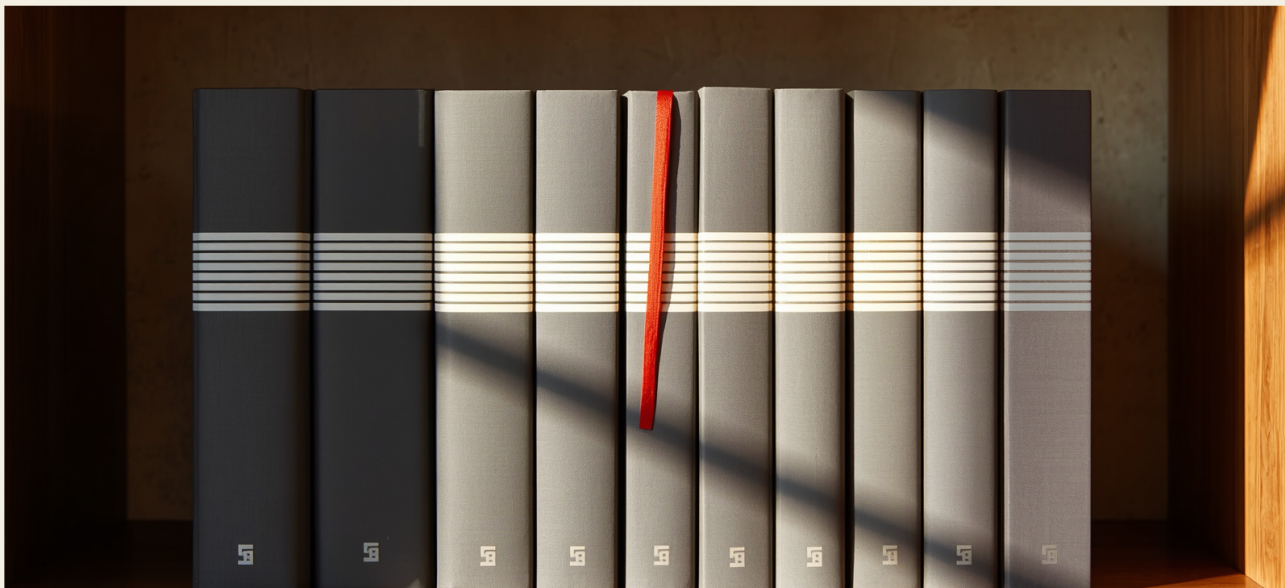
**Keep the sub-agent's working file.** Most sub-agent frameworks let you persist the agent's intermediate scratchpad, not just the final summary. Read the scratchpad when the summary feels suspiciously clean. The good stuff is often in the rejected branches of the investigation.

## Who should care

- **Engineering teams shipping with agentic coding tools** (Claude Code, Cursor, Aider, Devin, etc.): the background sub-agent pattern is now generally available, and your team workflow should incorporate it within two weeks. The hand-off bottleneck just got smaller; use it.
- **Solo builders running Claude Code on side projects:** the plan-mode hand-off is the under-marketed feature most worth your time. Stop redoing the planning work every time you start a session. Save the plans, version them in git, re-feed them.
- **Tool builders working on agent SDKs:** the skill system is a forcing function. If your agent framework can't load and invoke a Claude-style skill manifest, build that compatibility now. The community library is going to have hundreds of skills within six months, and your framework's value drops significantly if it can't share that ecosystem.

The Code w/ Claude event was a quiet release event with one big feature (background sub-agents) buried under polish theater. That's actually a good signal. Frontier labs talk loudest about the things that aren't quite ready and quietest about the things that just shipped to production. The skills system, the durable plans, the background agents — those are all in production now. Build with them this week.

If you're putting agents into production and trying to figure out the multi-agent coordination patterns, that's the territory we covered in [Agent Memory: The 5 Patterns That Ship in Production](#). The decision tree for sub-agents — what to delegate, how to summarize back, when the parent agent should re-investigate — is the same problem regardless of whether you're using Claude Code or rolling your own.



ESSAY 04 · AI CODING

---

# The best books for building AI agents in 2026

*An honest list of the books worth reading if you're shipping AI agents to production this year. What each covers, where each falls short, and the gap none of them fills.*

Flowi Editorial

The AI agent literature in 2026 is in an awkward place. The frontier moves quarterly. Most published books are 18-24 months out of date by the time they hit Kindle. The good content lives in scattered blog posts, conference talks, and a small set of recent books that have aged surprisingly well.

This is the honest list. Six books worth your reading time if you're shipping agents to production right now. What each is good for, where each is dated, and the gap none of them fills.

## The criteria

A book qualifies for this list if it answers at least one of:

1. **What's the architecture pattern** for an agent that actually ships?
2. **What's the failure mode** for that pattern, and how to engineer around it?
3. **What's the right mental model** for the underlying transformer / agent / tool-use system?

Books that are general "AI for business" overviews don't qualify — they're outdated the day they print. Books that are pure research surveys don't qualify — they're useful but they don't help you ship.

## The six books

1. **"Designing Machine Learning Systems"** — Chip Huyen, O'Reilly, 2022. Still the strongest single book on production ML systems generally. Agents are a subset.
2. **"Building LLM Applications for Production"** — Suhas Pai, Manning, 2024. The most current general overview; the production patterns chapter is the best part.
3. **"AI Engineering"** — Chip Huyen, O'Reilly, 2025. The follow-up to Designing ML Systems, written for the LLM era. Less generally-applicable than its predecessor but more relevant to agents specifically.
4. **"Building Multimodal AI Applications"** — Tarun Sharma, Packt, 2025. The only book that genuinely covers the multimodal patterns (vision-language-tool agents) honestly. Some chapters are thinner than others.
5. **"Hands-On Large Language Models"** — Jay Alammar & Maarten Grootendorst, O'Reilly, 2024. The best mental-model book — explains what's happening inside the transformer well enough that you stop being confused by edge cases.

6. **"Agent Memory: The 5 Patterns That Ship in Production"** — Flowi Editorial, 2026. Disclosure: we wrote this. The first book focused specifically on the memory layer that breaks at message four.

## "Designing Machine Learning Systems" — Chip Huyen

**The case for:** Even three years out, the framework for thinking about production ML is unmatched. The chapters on data engineering, feature stores, and monitoring apply directly to agent systems with minimal translation.

**The case against:** Pre-LLM era. The model-training emphasis is overweighted relative to where agent work actually lives now.

**Best for:** Engineers transitioning from traditional ML into LLM/agent work. The mental models transfer well; this book is the bridge.

## "Building LLM Applications for Production" — Suhas Pai

**The case for:** Solid coverage of RAG, tool-use, and basic agent patterns. The chapter on evaluation is unusually honest about how hard it is.

**The case against:** Tries to cover too much. The depth on any specific pattern is shallow. By the time you're past the basics, you've outgrown the book.

**Best for:** Engineers shipping their first production LLM application. Skip if you've already done two or three.

## "AI Engineering" — Chip Huyen

**The case for:** The most current treatment of the production AI stack. Excellent treatment of agentic workflows, fine-tuning decisions, and the LLMOps tooling landscape.

**The case against:** The agent-specific patterns chapter is good but not deep. You'll need to supplement with focused books or papers for any specific pattern.

**Best for:** Mid-level engineers building real systems, who want a textbook-level treatment of the stack.

## "Building Multimodal AI Applications" — Tarun Sharma

**The case for:** The vision-language sections are the best published treatment of the topic. Real code, real failure modes, honest about what's hard.

**The case against:** Chapters on speech and embodied agents are thinner. The book wants to be a complete multimodal reference but only really delivers on vision-language.

**Best for:** Engineers shipping agents that need to reason over images or screenshots. Skip the back half.

## "Hands-On Large Language Models" — Jay Alammar & Maarten Grootendorst

**The case for:** The best book for *understanding what's actually happening*. Alammar's visualization approach (he wrote "The Illustrated Transformer") translates to book-length and clarifies a lot of confusion.

**The case against:** Mental-model book, not a shipping book. You'll close it understanding the transformer better but not knowing more about production patterns.

**Best for:** Engineers who keep getting confused by edge cases ("why did the model do that?") and want the deeper intuition.

## "Agent Memory: The 5 Patterns That Ship in Production"

Disclosure: we wrote this. Take the recommendation with that in mind.

**The case for:** The only book focused entirely on the memory layer. The 5 patterns (Conversation Memory, Summary Memory, Vector Memory, Knowledge Graph Memory, Hybrid Memory) are the patterns you actually need; the failure modes named for production systems are the failure modes you'll hit. Short — 4,500 words, one weekend to read, code that runs.

**The case against:** Narrow scope by design. If you want a book that covers tools, planning, evaluation, *and* memory, this isn't it (yet — we're writing those).

**Best for:** Engineers whose agents work in demos but fail at message four. Specifically the *memory* failure mode. If your agent is failing at tool use or planning, this book won't help directly.



## The gap none of these fills

No book in the category covers **how to evaluate AI agents in production before they fail**. Every book has an "evaluation" chapter that's mostly theoretical — generic benchmarks, golden datasets, the classic ML stuff. None of them treat evaluation as the *operational discipline* it actually is — pre-deployment behavioral tests, ongoing regression suites, what to do when your eval breaks before your agent does.

This is where the next major book in the space needs to be written. We're working on one (book 04 in the Flowi roadmap). Until then, the gap lives in scattered blog posts, mostly from the Anthropic and OpenAI engineering teams, plus a few Substack writers (Eugene Yan, Hamel Husain, Simon Willison).

## How to pick

The honest decision tree by what you most care about:

- **Bridging from traditional ML** › Designing Machine Learning Systems
- **First-time LLM application** › Building LLM Applications for Production
- **Modern stack overview** › AI Engineering
- **Vision-language agents** › Building Multimodal AI Applications
- **Understanding the transformer** › Hands-On Large Language Models
- **Agent memory specifically** › Agent Memory: The 5 Patterns That Ship in Production

A recommendation: **don't read more than two books at once**. The category moves fast enough that depth matters more than breadth. Pick the one that addresses your current production bottleneck. Read it. Ship the thing. Pick the next one.

## Who should care

- **Engineers shipping their first agent:** start with Building LLM Applications for Production. Move to AI Engineering when you're past the basics.
- **Engineers whose agent works in demo but fails in production:** the issue is almost always memory, tools, or evaluation. Memory is what our book covers.

- **Solo and small-team builders:** budget book money. \$19-30 per book is rounding error for the time you save not figuring out what others have already figured out.

The AI agent book market is full of titles that don't survive the next quarter. The six above are the ones that have aged or will age — pick by your specific gap.

If your agent forgets the user at message four — which is the most consistent failure mode across every "AI agent demo" — that's the territory [Agent Memory: The 5 Patterns That Ship in Production](#) was built for. The decision tree, the code, and the failure modes nobody warns you about. \$19, 4,500 words, one weekend, ships on Monday.

THE DISPATCH · FREE · MONTHLY

# More like this, every morning.

The Flowi Daily Brief decodes one new AI release every day — what shipped, why it matters, what to do by Friday. Free monthly Dispatch arrives in your inbox.

[useflowi.app/dispatch](https://useflowi.app/dispatch)